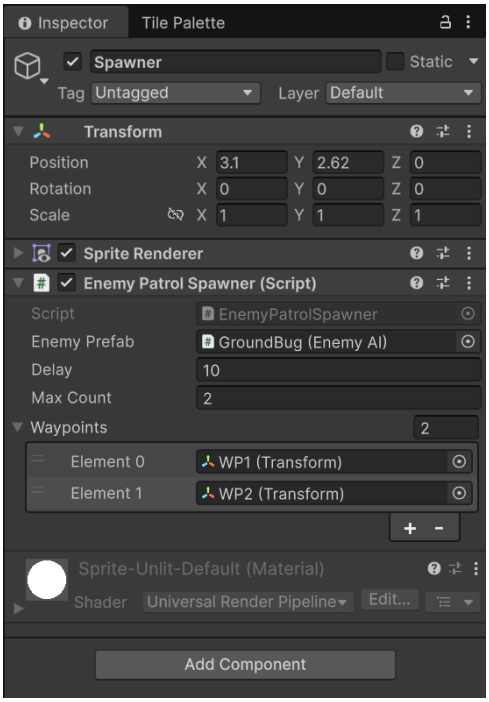
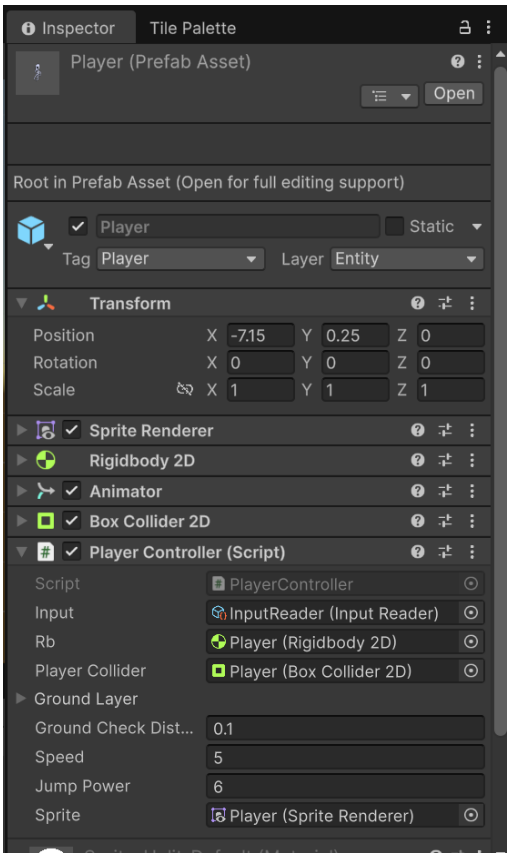
ПРАКТИКА 2

1 Использование систем: Transform; Rigidbody и Collider;

Transform используется для задания координат точек патрулирования.



Rigidbody и Collider используются в контроллере игрока.

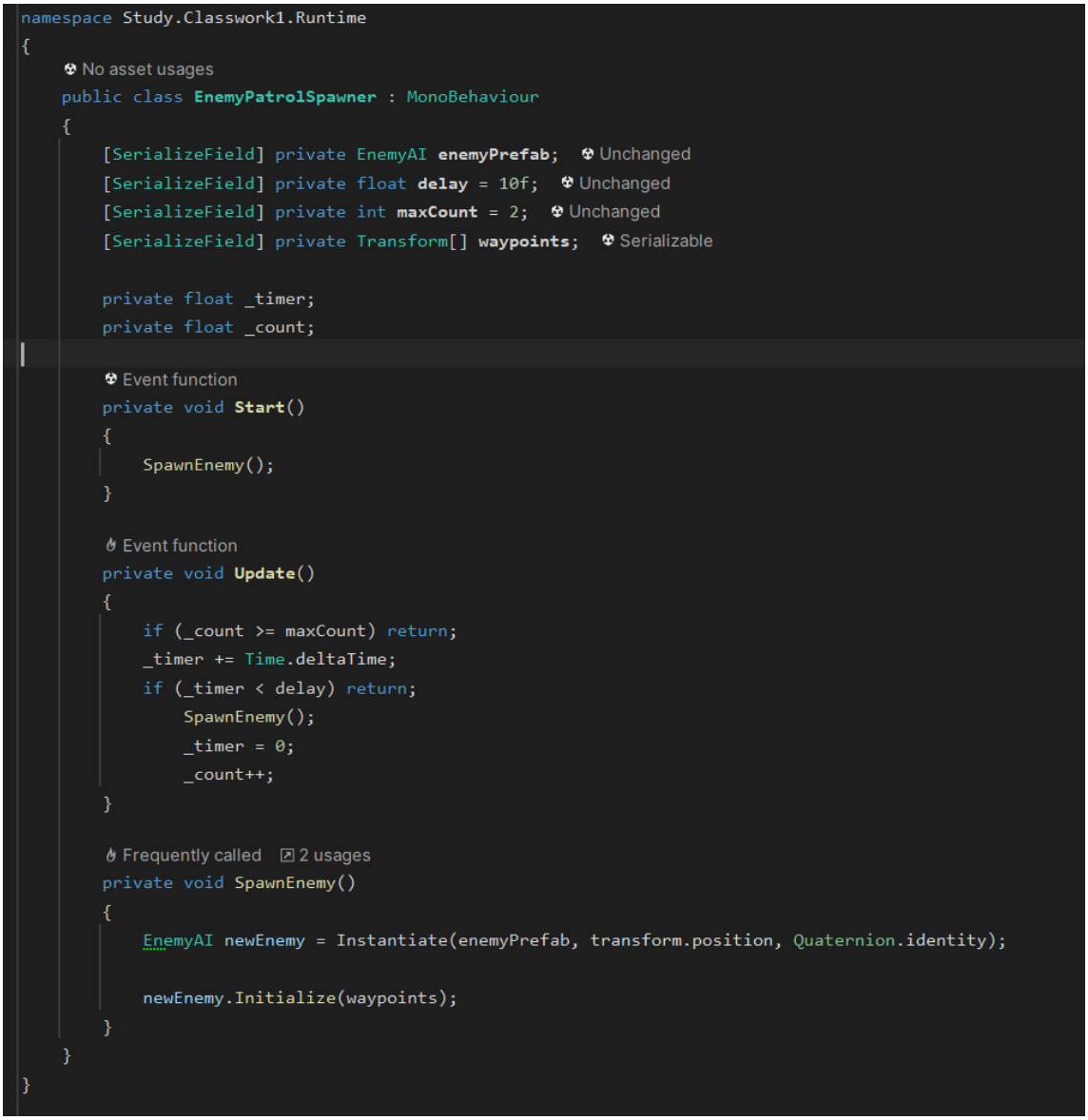


2 Создание пользовательского контроллера.

```
namespace Study.Classwork1.Runtime
{
    ☢ 1 asset usage
    public class PlayerController : MonoBehaviour
    {
        [SerializeField] private InputReader input;  ☢ InputReader.asset
        [SerializeField] private Rigidbody2D rb;  ☢ Changed in 1 asset
        [SerializeField] private Collider2D playerCollider;  ☢ Changed in 1 asset
        [SerializeField] private ContactFilter2D groundLayer;  ☢ Serializable
        [SerializeField] private float groundCheckDistance;  ☢ "0.1"
        [SerializeField] private float speed;  ☢ "5"
        [SerializeField] private float jumpPower;  ☢ "6"

        [SerializeField] private SpriteRenderer sprite;  ☢ Changed in 1 asset

        private bool _isMoving;
        private float _moveDirection;
        private readonly RaycastHit2D[] _groundHits = new RaycastHit2D[10];

        private bool _looksRight = true;

        ☢ Event function
        private void OnEnable()
        {
            input.MoveEvent += HandleMove;
            input.JumpEvent += HandleJump;
        }

        ☢ Event function
        private void OnDisable()
        {
            input.MoveEvent -= HandleMove;
            input.JumpEvent -= HandleJump;
        }

        ↗ 2 usages
        private void HandleMove(float direction, bool isPressed)
        {
            _isMoving = isPressed;
            _moveDirection = direction;
            if (!_isMoving) return;
            _looksRight = direction > 0;
            TurnHorizontal(_looksRight);
        }
```

```
        Event function
        private void FixedUpdate()
        {
            rb.linearVelocity = new Vector2(_moveDirection * speed, rb.linearVelocity.y);
        }

        ↗ 1 usage
        private void TurnHorizontal(bool right)
        {
            sprite.flipX = !right;
        }

        ↗ 2 usages
        private void HandleJump()
        {
            if (!IsGrounded()) return;
            rb.AddForce(Vector2.up * jumpPower, ForceMode2D.Impulse);
        }

        ↗ 1 usage
        private bool IsGrounded()
        {
            var hitsAmount :int  = playerCollider.Cast(Vector2.down, groundLayer, _groundHits, groundCheckDistance);
            return hitsAmount > 0;
        }

        ☢ Event function
        private void OnDrawGizmosSelected()
        {
            Gizmos.color = Color.■red;
            var bottom = new Vector3(playerCollider.bounds.center.x, playerCollider.bounds.min.y, 0);
            Gizmos.DrawLine(bottom, bottom + Vector3.down * groundCheckDistance);
        }
    }
```

3 Работа с переменными персонажа; Создание навигационной системы и объекта противника или NPC.



```
namespace Study.Classwork1.Runtime
{
    No asset usages
    public class EnemyPatrolSpawner : MonoBehaviour
    {
        [SerializeField] private EnemyAI enemyPrefab;   Unchanged
        [SerializeField] private float delay = 10f;   Unchanged
        [SerializeField] private int maxCount = 2;   Unchanged
        [SerializeField] private Transform[] waypoints;   Serializable

        private float _timer;
        private float _count;

        Event function
        private void Start()
        {
            SpawnEnemy();
        }

        Event function
        private void Update()
        {
            if (_count >= maxCount) return;
            _timer += Time.deltaTime;
            if (_timer < delay) return;
                SpawnEnemy();
                _timer = 0;
                _count++;
        }

        Frequently called   2 usages
        private void SpawnEnemy()
        {
            EnemyAI newEnemy = Instantiate(enemyPrefab, transform.position, Quaternion.identity);

            newEnemy.Initialize(waypoints);
        }
    }
}
```

Рисунок 1 – Спавнер врагов с патрулированием

```
1 asset usage  2 usages
public class EnemyAI : MonoBehaviour
{
    [SerializeField] private float speed = 3f;  Unchanged
    [SerializeField] private Rigidbody2D rb;  Changed in 1 asset

    private Transform[] _waypoints;

    private int _currentPointIndex;

    Frequently called  1 usage
    public void Initialize(Transform[] waypoints)
    {
        _waypoints = waypoints;
    }

    Event function
    private void FixedUpdate()
    {
        if (_waypoints.Length == 0) return;

        Patrol();
    }
```

Рисунок 2 – ИИ врагов с патрулированием, часть 1

```
    Frequently called  1 usage
    private void Patrol()
    {
        var target :Transform = _waypoints[_currentPointIndex];

        var direction :Vector2 = new Vector2(target.position.x - transform.position.x, 0).normalized;

        rb.linearVelocity = new Vector2(direction.x * speed, rb.linearVelocity.y);

        transform.localScale = direction.x switch
        {
            > 0 => new Vector3(1, 1, 1),
            < 0 => new Vector3(-1, 1, 1),
            _ => transform.localScale
        };

        var distanceToTarget :float = Mathf.Abs( f: target.position.x - transform.position.x);

        if (distanceToTarget < 0.2f)
        {
            _currentPointIndex++;

            if (_currentPointIndex >= _waypoints.Length)
            {
                _currentPointIndex = 0;
            }
        }
    }
}
```

Рисунок 3 – ИИ врагов с патрулированием, часть 2